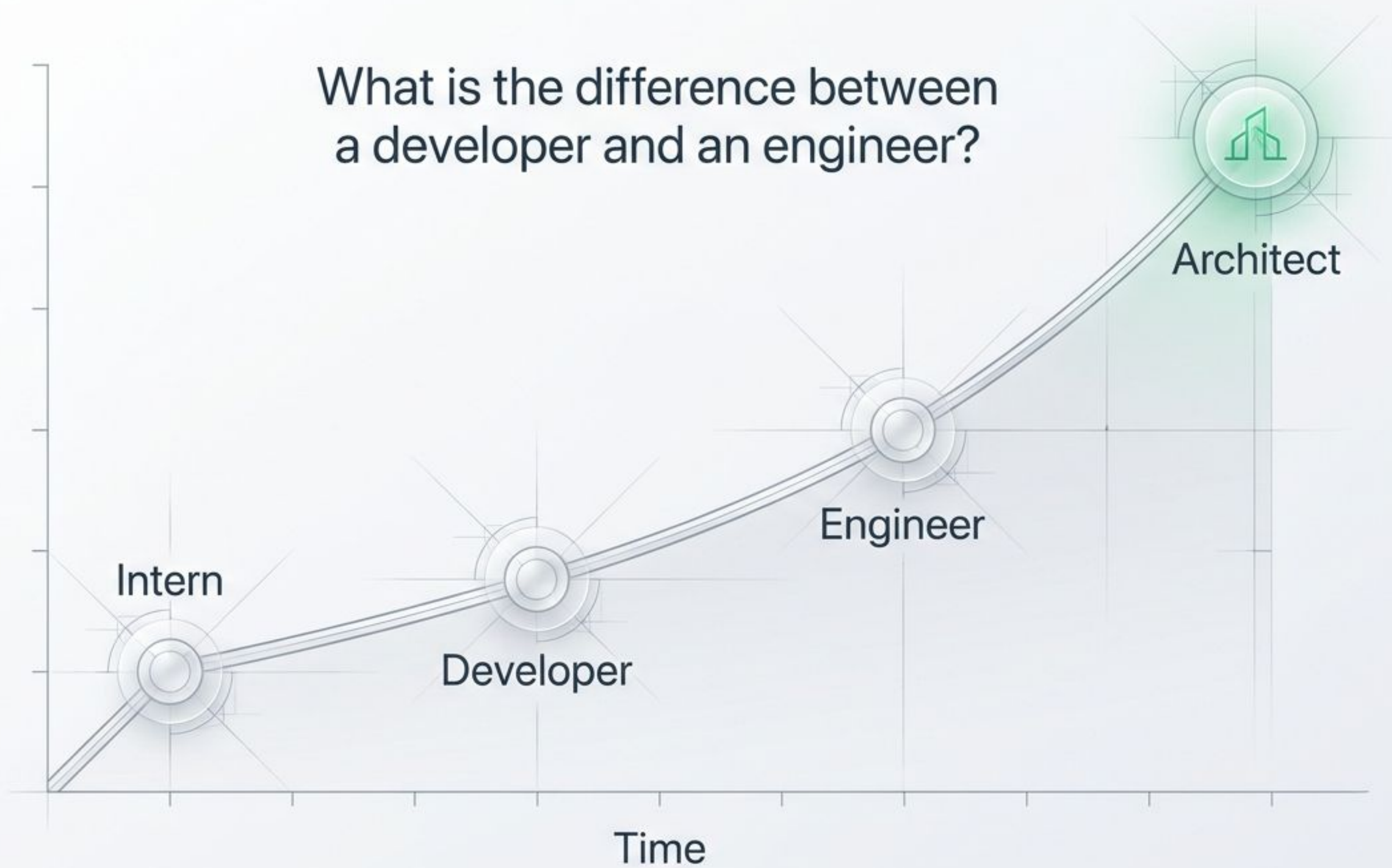


Vanakam Makkale,
Namaskara Bengaluru

What is the difference between a developer and an engineer?

Problem Solving Scale

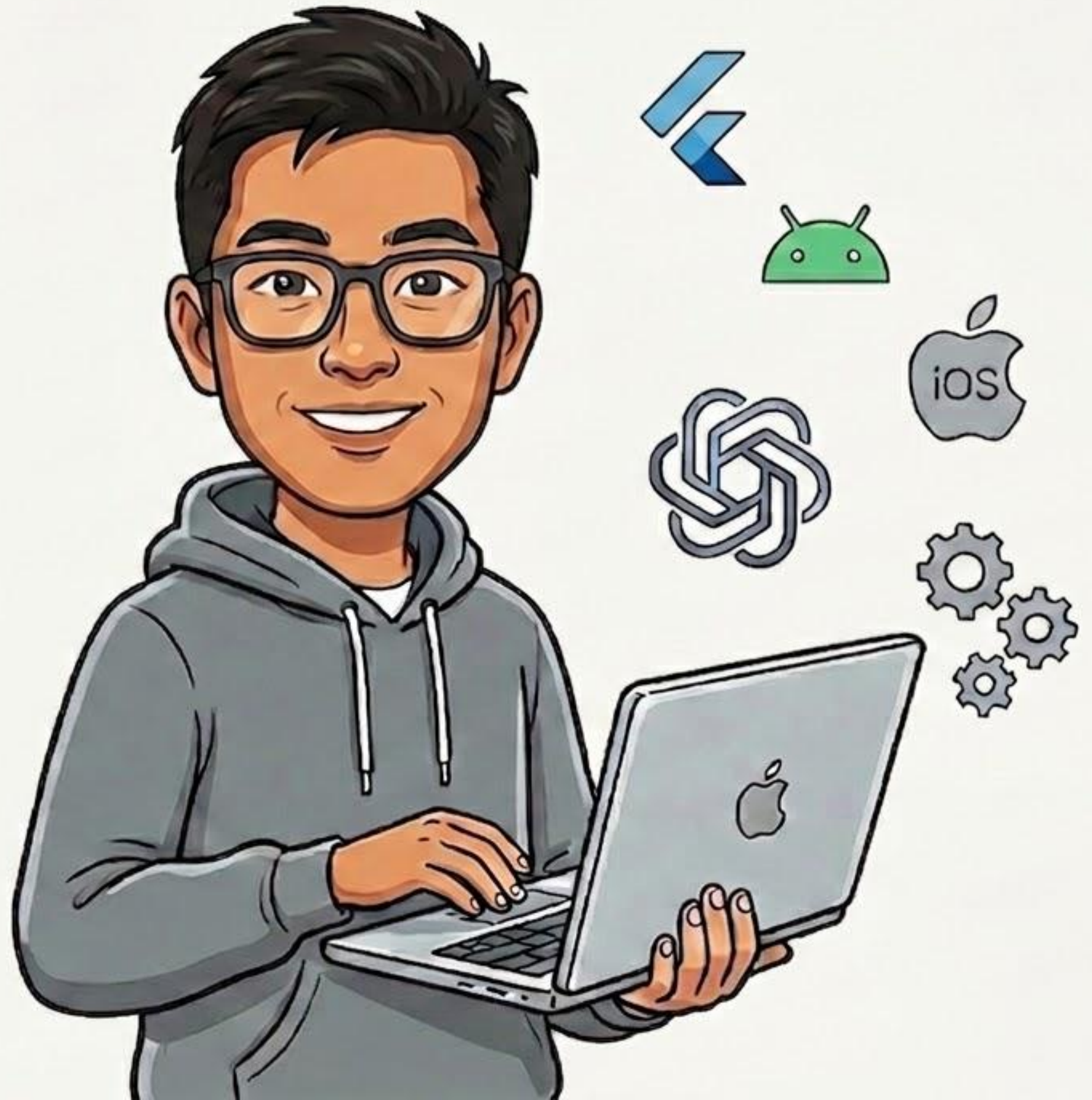


**YOUR FLUTTER SECRETS
ARE NOT SECRET**

Akash Senthil

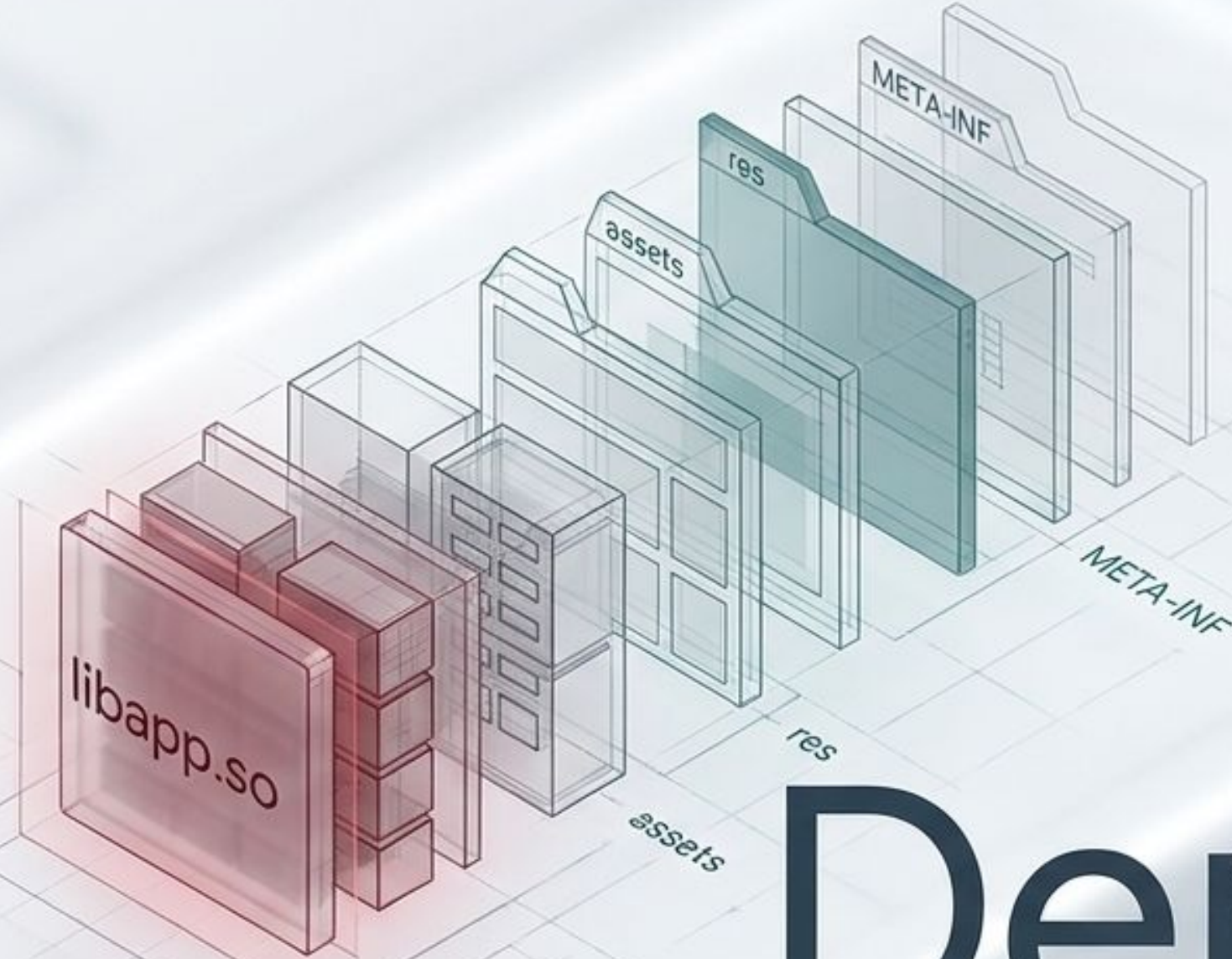
Product Engineer @Samaaro | Flutter Developer (4+ Years Experience)

Member of Namma Flutter | Flutter Chennai Community



The illusion of environment security.

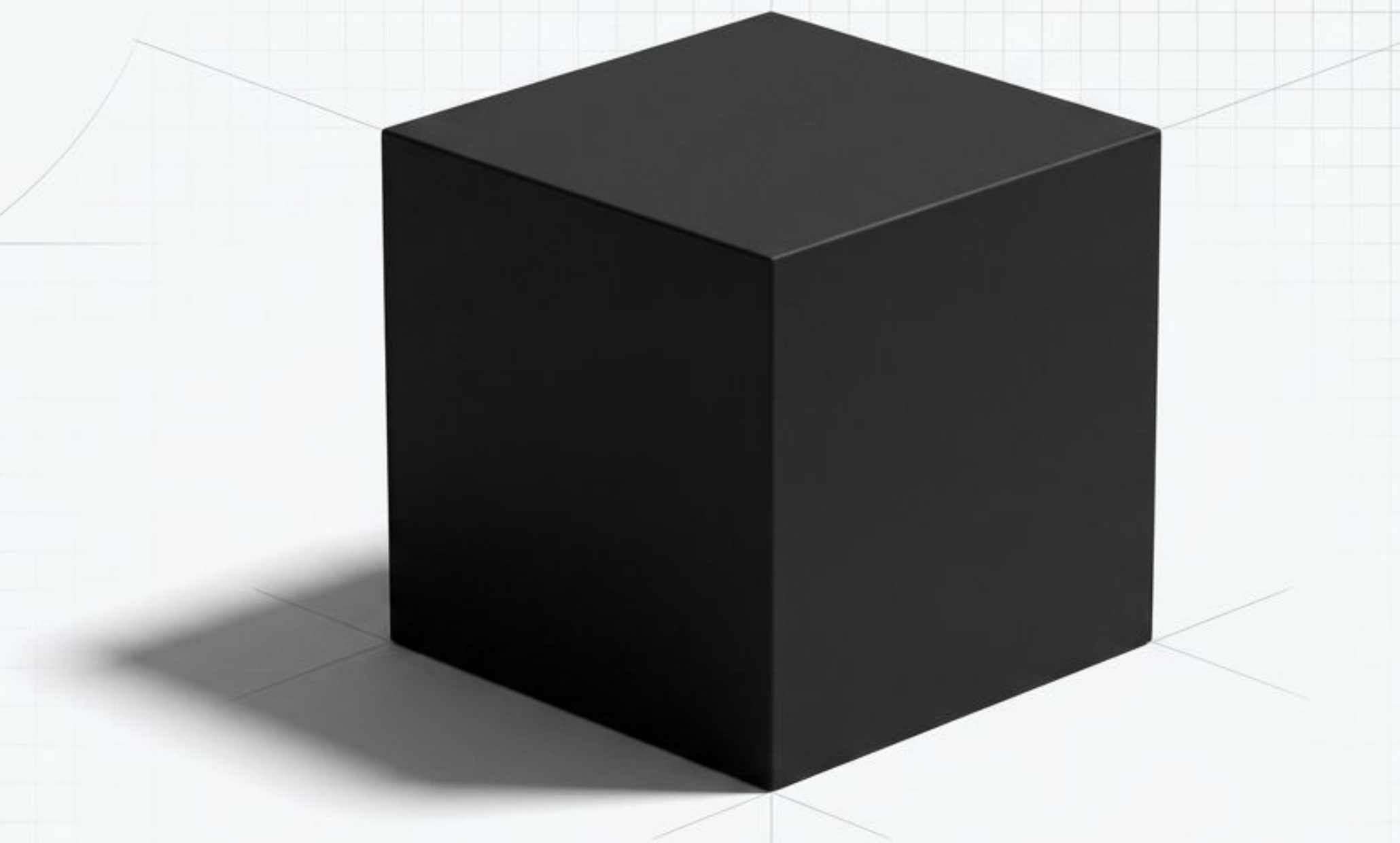




libapp.so

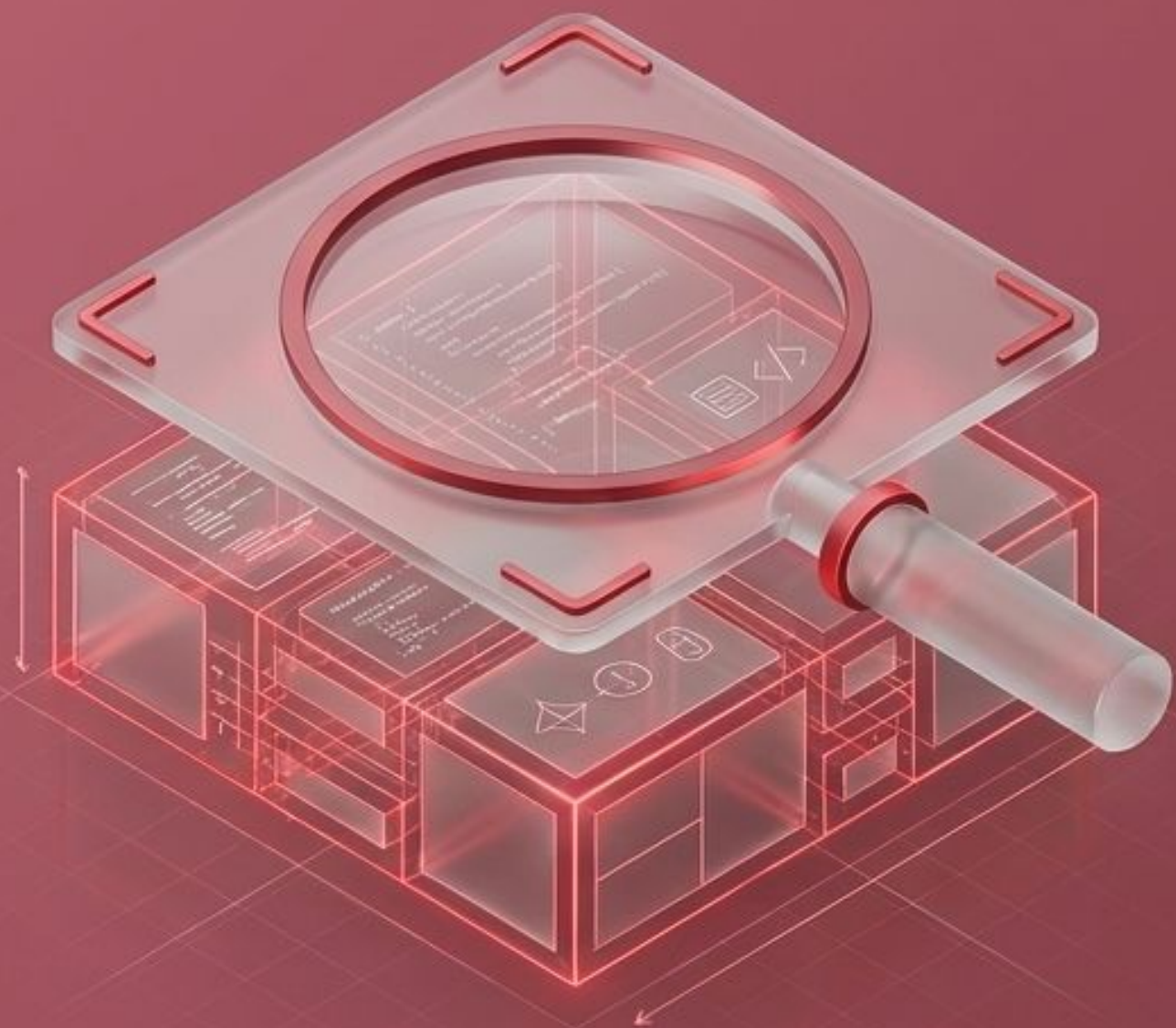
The diagram illustrates a 3D file system structure. On the left, a red folder labeled 'libapp.so' is shown. To its right is a series of blue folders arranged in a row, labeled 'assets', 'res', and 'META-INF'. The 'assets' folder is further subdivided into smaller blue blocks. The entire structure is set against a background of a light blue grid with faint, glowing lines.

Demo



Black box

Reverse Engineering Landscapes



Attack 1 — Static Analysis

Extracting hardcoded URLs, API keys, and configurations directly from the binary structure.

The Reality

How many hardcoded secrets exist
in your application today?

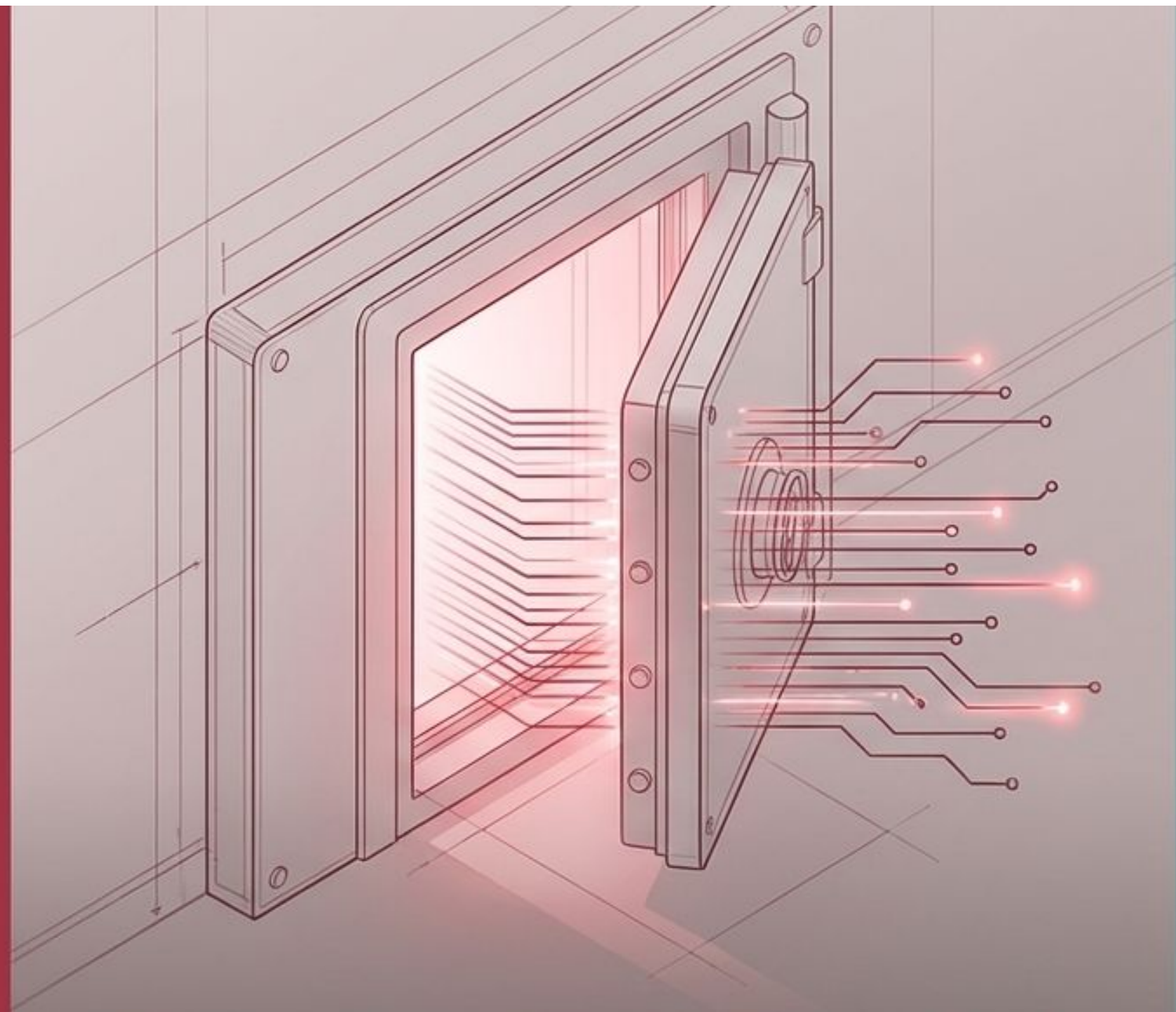


Attack 2 — Dart AOT Analysis

Extracting string literals and configurations directly from libapp.so.

The Reality

Obfuscation protects names, not knowledge. It is not encryption.



Attack 3 — Runtime Analysis

Bypassing Flutter SecureStorage, SSL Pinning, and Runtime APIs.

The Reality

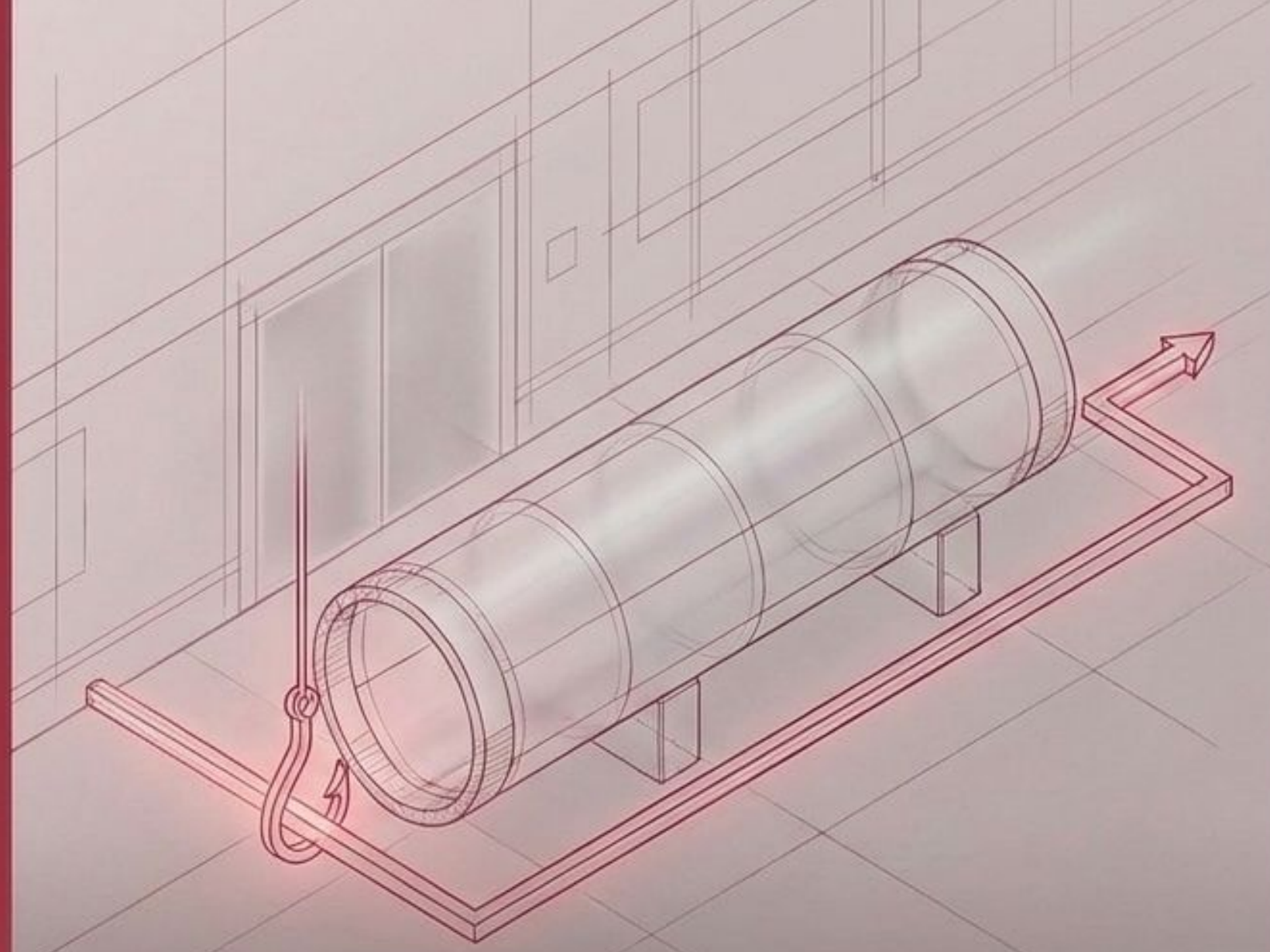
Encryption protects data at rest. It cannot hide data after decryption in memory.

Attack 4 — Brute Force & API Analysis

What happens after an attacker understands your API structure?

The Defense

Never trust the client. Every request can be perfectly reproduced outside your app.

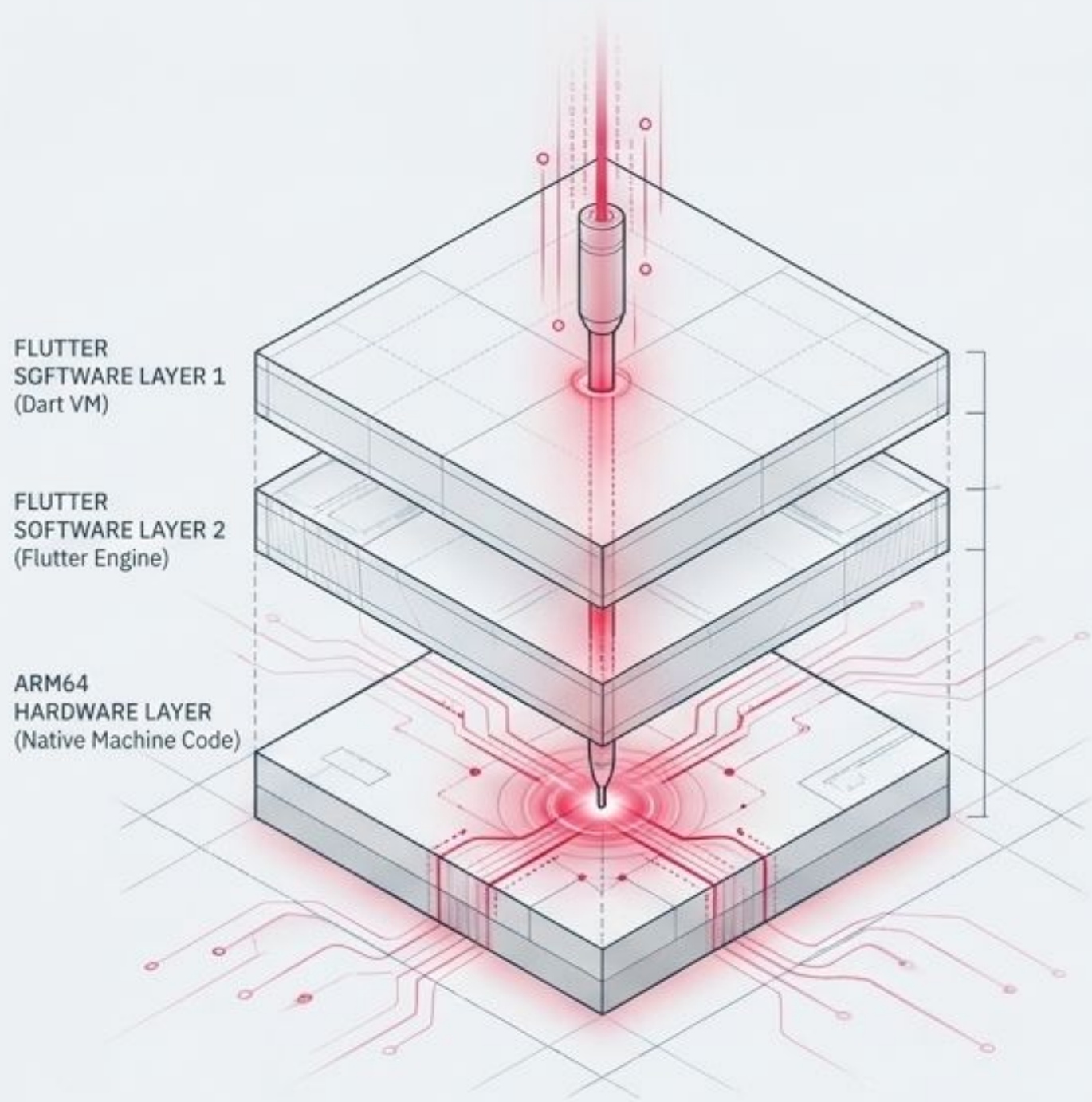


Attack 5 — SSL Pinning Bypass

Attackers controlling trust decisions via Frida hooks, BoringSSL, and native bypasses.

The Reality

SSL Pinning protects the connection, not a compromised device. It must be only one layer of defense.



The diagram illustrates the architecture of Flutter on ARM64 hardware. It consists of three stacked layers: the top layer is 'FLUTTER SOFTWARE LAYER 1 (Dart VM)', the middle is 'FLUTTER SOFTWARE LAYER 2 (Flutter Engine)', and the bottom is 'ARM64 HARDWARE LAYER (Native Machine Code)'. A red probe is shown penetrating through the top two layers and reaching the hardware layer, with red lines indicating data flow or analysis paths. The hardware layer is depicted with circuit-like patterns.

FLUTTER
SOFTWARE LAYER 1
(Dart VM)

FLUTTER
SOFTWARE LAYER 2
(Flutter Engine)

ARM64
HARDWARE LAYER
(Native Machine Code)

Attack 6 – ARM64 Native Analysis

Discarding Flutter tools to analyse native machine code directly.

The Reality

Native code raises the bar significantly, but it does not make reverse engineering impossible.

Prevention Strategies



Never Embed
Private Keys



Server-Side
Authentication



Protect &
Encrypt Assets



Build with
Obfuscation



Detect
Rooted/Jailbroken
Devices

freerasp, flutter_jailbreak_detection



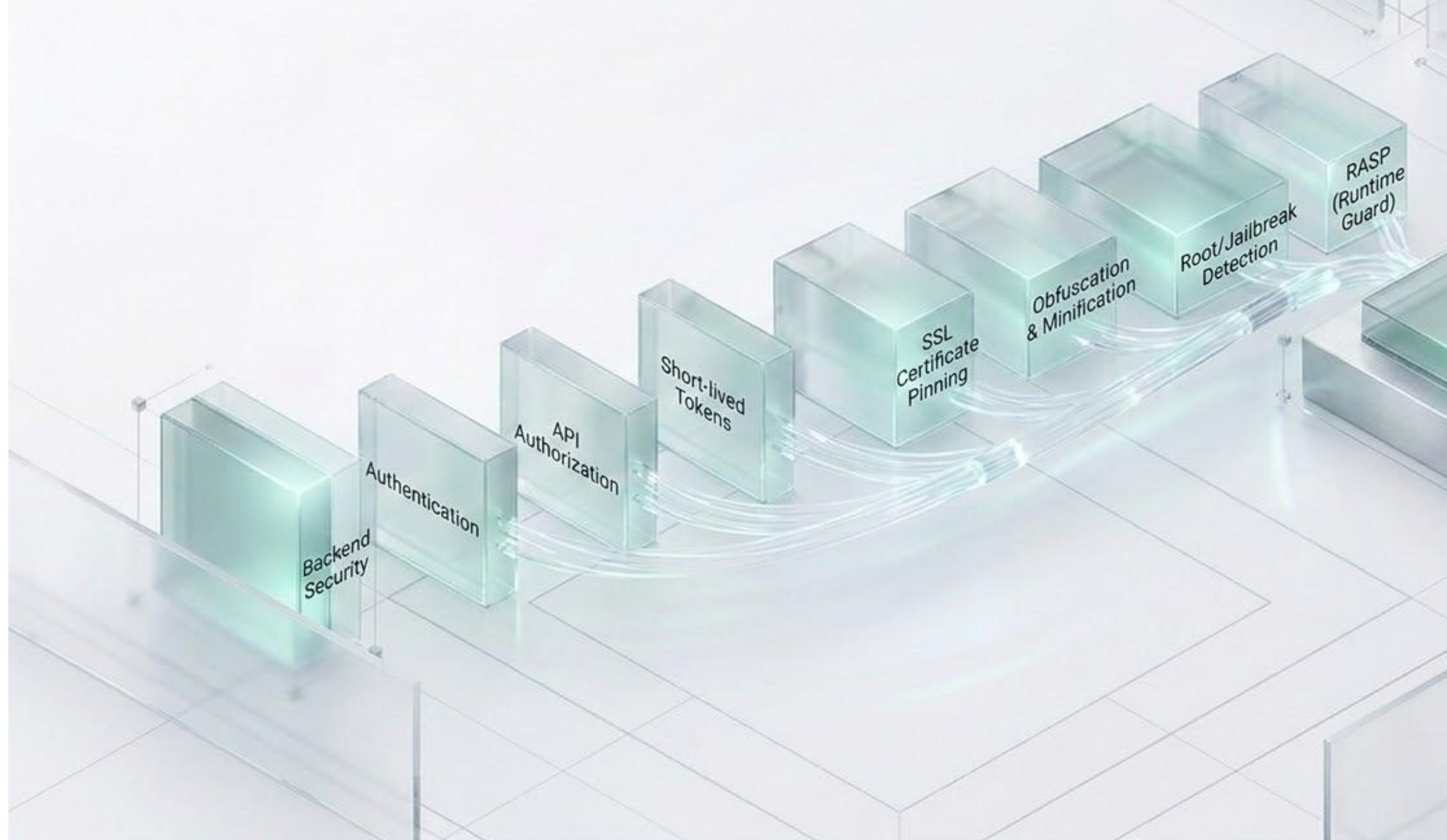
Use RASP for
High-Risk Apps



Fetch Config
from Backend



Minimize
Binary Content



~~Obfuscation prevents
reverse engineering.~~

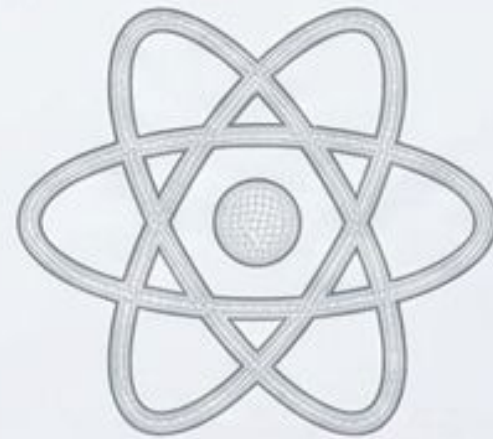
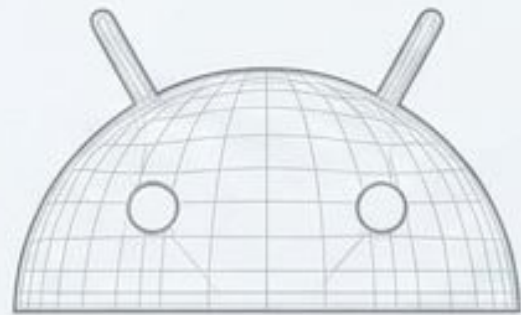
~~Flutter SecureStorage
protects my secrets.~~

~~SSL Certificate Pinning
makes my API~~
impossible to intercept.

~~100% Secured.~~

**YOUR FLUTTER SECRETS
ARE NOT SECRET**

**Architectural resilience applies
universally across all mobile frameworks.**





Read the complete research.



Connect and validate.

The background of the slide is a light gray architectural drawing. It features a central circular area with a soft, multi-colored gradient (yellow, green, blue, and pink). Overlaid on this circle is the text 'Q/A' in a large, bold, black sans-serif font. The background also includes faint, thin-lined architectural sketches of buildings and a grid pattern in the corners.

Q/A

Thanks, everyone.